

ASSIGNMENT 12.4

NAME:-PRATHAMKOUSHIK

ROLLNO:-2403A52182

Task 1: Implementing Bubble Sort with AI Comments

- Task: Write a Python implementation of Bubble Sort.
- Instructions:
 - o Students implement Bubble Sort normally.
 - o Ask AI to generate inline comments explaining key logic (like swapping, passes, and termination).
 - o Request AI to provide time complexity analysis.
- Expected Output:
 - o A Bubble Sort implementation with AI-generated explanatory comments and complexity analysis

```
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all array elements
    for i in range(n):
        # Last i elements are already in place
        for j in range(0, n-i-1):
            # traverse the array from 0 to n-i-1
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j] # Swapping elements

    return arr # Return the sorted array

# Example usage:
my_list = [64, 34, 25, 12, 22, 11, 90]
sorted_list = bubble_sort(my_list)
print("Sorted array is:", sorted_list)
```

Sorted array is: [11, 12, 22, 25, 34, 64, 90]

Time Complexity Analysis of Bubble Sort

Bubble sort has a time complexity of $O(n^2)$ in the worst and average cases. This is because in the worst case (when the array is in reverse order), the algorithm needs to make $n-1$ passes, and in each pass, it makes up to $n-1$ comparisons and swaps.

In the best case (when the array is already sorted), bubble sort has a time complexity of $O(n)$. This is because the algorithm only needs to make one pass to confirm that the array is sorted after the first pass if no swaps are made.

S give me the
expanding
complexity

Space Complexity Analysis of Bubble Sort

Bubble sort has a space complexity of $O(1)$. This means that the amount of extra memory used by the algorithm is constant, regardless of the size of the input array.

The algorithm only uses a few extra variables for temporary storage during the swapping process. It does not require any additional data structures that grow with the size of the input.

Task 2: Optimizing Bubble Sort → Insertion Sort

- Task: Provide Bubble Sort code to AI and ask it to suggest a more efficient algorithm for partially sorted arrays.
- Instructions:
 - o Students implement Bubble Sort first.
 - o Ask AI to suggest an alternative (Insertion Sort).
 - o Compare performance on nearly sorted input.
- Expected Output:
 - o Two codes (Bubble Sort + Insertion Sort).
 - o AI explanation of why Insertion Sort is more efficient for

partially sorted data.

```
def insertion_sort(arr):  
    # Iterate through the input array starting from the second element  
    for i in range(1, len(arr)):  
        # Store the current element in a temporary variable called key  
        key = arr[i]  
        # Initialize a variable j to the index of the element just before i  
        j = i - 1  
        # While j is greater than or equal to 0 and the element at index j is greater than key,  
        # shift the element at index j one position to the right (j = j + 1)  
        while j >= 0 and key < arr[j]:  
            arr[j + 1] = arr[j]  
            j -= 1  
        # After the while loop, place the key at the correct sorted position  
        arr[j + 1] = key  
    # After the outer loop finishes, return the sorted array arr  
    return arr  
  
# Example usage:  
my_list = [12, 11, 13, 5, 6]  
sorted_list = insertion_sort(my_list)  
print("Sorted array is:", sorted_list)
```

Sorted array is: [5, 6, 11, 12, 13]

Compare Performance

Subtask:

Compare the execution time of Bubble Sort, Merge Sort, Quick Sort, and Insertion Sort

```
import time
import random

# We need the functions for merge_sort and quick_sort.
# Since these were provided in previous turns, let's assume they are available.
# If they are not, you would need to regenerate them here or ensure they are.

# Function to generate a nearly sorted array
def generate_nearly_sorted_array(size, num_swaps):
    arr = list(range(size))
    for _ in range(num_swaps):
        i, j = random.sample(range(size), 2)
        arr[i], arr[j] = arr[j], arr[i]
    return arr

# Define the sorting algorithms to compare
sorting_algorithms = {
    "Bubble Sort": bubble_sort,
    "Insertion Sort": insertion_sort,
    "Merge Sort": merge_sort,
    "Quick Sort": quick_sort
}

# Generate a nearly sorted array
array_size = 5000
num_swaps = 50
nearly_sorted_array = generate_nearly_sorted_array(array_size, num_swaps)

# Compare the performance of each algorithm
results = {}
for name, algorithm in sorting_algorithms.items():
    # Create a copy of the nearly sorted array for each algorithm
    arr_copy = nearly_sorted_array[:]
    start_time = time.time()
    algorithm(arr_copy)
    end_time = time.time()
    execution_time = end_time - start_time
    results[name] = execution_time
    print(f'{name}: {execution_time:.6f} seconds')

# Print the results
print("\nPerformance comparison on a nearly sorted array:")
for name, execution_time in results.items():
    print(f'{name}: {execution_time:.6f} seconds')
```

```
Bubble Sort: 0.518712 seconds
Insertion Sort: 0.012460 seconds
Merge Sort: 0.006346 seconds
```

Explain Efficiency

Insertion sort is efficient for partially sorted arrays because of its adaptive nature. Here's why:

1. **Fewer Swaps and Comparisons:** Insertion sort works by inserting each element into its correct position within a sorted portion of the array. In a nearly sorted array, elements are already close to their final positions. The inner loop (which shifts elements to make space for the current element) will run fewer times for each element, leading to fewer swaps and comparisons, which leads to faster execution.
2. **$O(n)$ in Best Case:** When the array is already sorted (the best-case scenario, which is a form of a nearly sorted array), the while loop condition `key < arr[j]` will rarely be true. As a result, the algorithm only performs a single comparison and no swaps for each element, resulting in a time complexity of $O(n)$. For nearly sorted arrays, the performance is close to this best case.
3. **Lower Overhead:** Compared to more complex algorithms like Merge Sort or Quick Sort, Insertion Sort has a simpler implementation and is an in-place sorting algorithm (it doesn't require extra space for merging or partitioning). This makes it faster for smaller arrays or arrays that are already mostly sorted.

In contrast, algorithms like Bubble Sort always perform $O(n^2)$ comparisons and swaps regardless of the data. While an optimization (like a flag to stop early) is added to stop early, and algorithms like Merge Sort and Quick Sort have a more consistent $O(n \log n)$ performance regardless of whether the data is partially sorted or not. While $O(n \log n)$ is generally better for large, unsorted data, the $O(n)$ behavior of insertion sort on nearly sorted data makes it outperform them in that specific scenario.

Task 3: Binary Search vs Linear Search

- Task: Implement both Linear Search and Binary Search.
- Instructions:
 - o Use AI to generate docstrings and performance notes.
 - o Test both algorithms on sorted and unsorted data.
 - o Ask AI to explain when Binary Search is preferable.
- Expected Output:
 - o Two implementations with docstrings.
 - o A student observation table comparing performance (Linear vs Binary Search)

```
def linear_search(arr, target):
    """
    Performs a linear search on a list to find the target value.

    Args:
        arr: The list to search within.
        target: The value to search for.

    Returns:
        The index of the target value if found, otherwise -1.
    """
    for index, value in enumerate(arr):
        if value == target:
            return index
    return -1

# Example usage:
my_list = [10, 5, 8, 12, 3]
target_value = 8
index = linear_search(my_list, target_value)
print(f"The index of {target_value} is: {index}")

target_value = 100
index = linear_search(my_list, target_value)
print(f"The index of {target_value} is: {index}")
```

→ The index of 8 is: 2
The index of 100 is: -1

```

def binary_search(arr, target):
    """
    Performs a binary search on a sorted list to find the index of the target value.

    Args:
        arr: The sorted list to search within.
        target: The value to search for.

    Returns:
        The index of the target value if found, otherwise -1.

    Note:
        The input list 'arr' must be sorted for binary search.
    """
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2
        mid_val = arr[mid]

        if mid_val == target:
            return mid
        elif mid_val < target:
            low = mid + 1
        else: # mid_val > target
            high = mid - 1

    return -1

# Example usage:
sorted_list = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
target_value_found = 23
index_found = binary_search(sorted_list, target_value_found)
print(f"The index of {target_value_found} is: {index_found}")

target_value_not_found = 50
index_not_found = binary_search(sorted_list, target_value_not_found)
print(f"The index of {target_value_not_found} is: {index_not_found}")

```

```

The index of 23 is: 5
The index of 50 is: -1

```



```

import random

def generate_sorted_list(size):
    """
    Generates a sorted list of integers from 0 to size-1.

    Args:
        size: The number of elements in the list.

    Returns:
        A sorted list of integers.
    """
    return list(range(size))

def generate_unsorted_list(size):
    """
    Generates a list of random integers.

    Args:
        size: The number of elements in the list.

    Returns:
        An unsorted list of random integers.
    """
    return [random.randint(0, size * 2) for _ in range(size)]

# Example usage:
sorted_list_example = generate_sorted_list(10)
print("Example sorted list:", sorted_list_example)

unsorted_list_example = generate_unsorted_list(10)
print("Example unsorted list:", unsorted_list_example)

```


8
 Example sorted list: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
 Example unsorted list: [10, 4, 8, 9, 8, 13, 0, 4, 5, 6]


```

# Generate test data
sorted_list = generate_sorted_list(size)
unsorted_list = generate_unsorted_list(size)
target_value = size // 2 # A value likely to be in the list

# Measure Linear Search on Unsorted Data
start_time = time.time()
linear_search(unsorted_list, target_value)
end_time = time.time()
linear_unsorted_time = end_time - start_time
print(f" Linear Search (Unsorted): {linear_unsorted_time} seconds")

# Measure Linear Search on Sorted Data
start_time = time.time()
linear_search(sorted_list, target_value)
end_time = time.time()
linear_sorted_time = end_time - start_time
print(f" Linear Search (Sorted): {linear_sorted_time} seconds")

# Measure Binary Search on Sorted Data
start_time = time.time()
binary_search(sorted_list, target_value)
end_time = time.time()
binary_sorted_time = end_time - start_time
print(f" Binary Search (Sorted): {binary_sorted_time} seconds")

```

Testing with list size: 1000

```

Linear Search (Unsorted): 0.000059 seconds
Linear Search (Sorted): 0.000024 seconds
Binary Search (Sorted): 0.000007 seconds

```

Testing with list size: 10000

```

Linear Search (Unsorted): 0.000568 seconds
Linear Search (Sorted): 0.000455 seconds
Binary Search (Sorted): 0.000006 seconds

```

Testing with list size: 100000

```
# This is a markdown cell.
```

# Feature	Linear Search	Binary Search
# -----	-----	-----
# Time Complexity (Best Case)	$O(1)$	$O(1)$
# Time Complexity (Average Case)	$O(n)$	$O(\log n)$
# Time Complexity (Worst Case)	$O(n)$	$O(\log n)$
# Data Requirement	Unsorted or Sorted	**Must be Sorted**
# Ease of Implementation	Simpler	More complex
# Performance on Small Datasets	Relatively fast	Relatively fast (over
# Performance on Large Datasets	Slow (becomes impractical)	Very fast
# Use Cases	Unsorted data, simple searches	Frequent searches on

Task 4: Quick Sort and Merge Sort Comparison

- Task: Implement Quick Sort and Merge Sort using recursion.
- Instructions:
 - o Provide AI with partially completed functions for recursion.
 - o Ask AI to complete the missing logic and add docstrings.
 - o Compare both algorithms on random, sorted, and reverse-sorted lists.
- Expected Output:
 - o Working Quick Sort and Merge Sort implementations.
 - o AI-generated explanation of average, best, and worst-case complexities

✓ Complete quick sort

Subtask:

Implement the quick sort algorithm with a docstring.

Reasoning: Implement the quick sort algorithm with a docstring as described

[14]

✓ Os



```
def quick_sort(arr):
```

```
    """Sorts a list using the Quick Sort algorithm.
```

```
    Args:
```

```
        arr: The list to be sorted.
```

```
    Returns:
```

```
        A new list containing the sorted elements.
```

```
    if len(arr) <= 1:
```

```
        return arr
```

```
    else:
```

```
        pivot = arr[-1] # Choose the last element as the pivot
```

```
        less = [x for x in arr[:-1] if x <= pivot]
```

```
        greater = [x for x in arr[:-1] if x > pivot]
```

```
        return quick_sort(less) + [pivot] + quick_sort(greater)
```

```
    # Example usage:
```

```
    my_list = [10, 7, 8, 9, 1, 5]
```

```
    sorted_list = quick_sort(my_list)
```

```
    print("Sorted array is:", sorted_list)
```

```
    my_list_2 = [3, 0, 1, 8, 7, 2, 5, 4, 6, 9]
```

```
    sorted_list_2 = quick_sort(my_list_2)
```

```
    print("Sorted array is:", sorted_list_2)
```



```
Sorted array is: [1, 5, 7, 8, 9, 10]
```

```
Sorted array is: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```

def merge_sort(arr):
    """
    Sorts a list using the Merge Sort algorithm.

    Args:
        arr: The list to be sorted.

    Returns:
        A new list containing the sorted elements.
    """
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left_half = arr[:mid]
    right_half = arr[mid:]

    left_half_sorted = merge_sort(left_half)
    right_half_sorted = merge_sort(right_half)

    return merge(left_half_sorted, right_half_sorted)

def merge(left, right):
    """
    Merges two sorted lists into a single sorted list.

    Args:
        left: The first sorted list.
        right: The second sorted list.

    Returns:
        A new sorted list containing elements from both input lists.
    """
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    # Append remaining elements from either list
    result.extend(left[i:])
    result.extend(right[j:])

    return result

# Example usage:
my_list = [38, 27, 43, 3, 9, 82, 18]
sorted_list = merge_sort(my_list)
print("Sorted array is:", sorted_list)

```

Generate test data

Subtask:

Create functions to generate random, sorted, and reverse-sorted lists of varying sizes for performance testing.

Reasoning: Define the functions to generate random, sorted, and reverse-sorted lists as required and provide example usage.

```
import random

def generate_random_list(size):
    """
    Generates a list of random integers.

    Args:
        size: The number of elements in the list.

    Returns:
        An unsorted list of random integers.
    """
    return [random.randint(0, size * 2) for _ in range(size)]

def generate_sorted_list(size):
    """
    Generates a sorted list of integers from 0 to size - 1.

    Args:
        size: The number of elements in the list.

    Returns:
        A sorted list of integers.
    """
    return list(range(size))

def generate_reverse_sorted_list(size):
    """
    Generates a reverse-sorted list of integers from size - 1 to 0.

    Args:
        size: The number of elements in the list.

    Returns:
        A reverse-sorted list of integers.
    """
    return list(range(size - 1, -1, -1))

# Example usage:
random_list_example = generate_random_list(10)
print("Example random list:", random_list_example)

sorted_list_example = generate_sorted_list(10)
print("Example sorted list:", sorted_list_example)

reverse_sorted_list_example = generate_reverse_sorted_list(10)
print("Example reverse-sorted list:", reverse_sorted_list_example)
```

Example random list: [0, 18, 12, 17, 1, 20, 7, 19, 5, 5]

Example sorted list: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

Compare performance

Subtask:

Write code to measure and compare the execution time of quick sort and merge sort on the

Reasoning: Measure and compare the execution time of quick sort and merge sort on the previously defined functions and the `time` module.

```
import time
import random

# Assuming quick_sort and merge_sort functions are defined in previous cells
# Assuming generate_random_list, generate_sorted_list, and generate_reverse_sort

list_sizes = [1000, 10000, 100000]

for size in list_sizes:
    print(f"\n--- Testing with list size: {size} ----")

    # Generate test data
    random_list = generate_random_list(size)
    sorted_list = generate_sorted_list(size)
    reverse_sorted_list = generate_reverse_sorted_list(size)

    test_data = {
        "Random": random_list,
        "Sorted": sorted_list,
        "Reverse Sorted": reverse_sorted_list
    }

    sorting_algorithms = {
        "Quick Sort": quick_sort,
        "Merge Sort": merge_sort
    }

    for data_type, data_list in test_data.items():
        print(f"\n Data Type: {data_type}")
        for algo_name, algorithm in sorting_algorithms.items():
            # Create a copy to avoid modifying the original list
            data_copy = data_list[:]

            start_time = time.time()
            # Call the sorting algorithm
            # Note: quick_sort returns a new list, merge_sort returns a new list
            if algo_name == "Quick Sort":
                sorted_data = algorithm(data_copy)
            else: # Merge Sort
                sorted_data = algorithm(data_copy)

            end_time = time.time()
            execution_time = end_time - start_time

            print(f"    {algo_name}: {execution_time:.6f} seconds")
```

Task 5: AI-Suggested Algorithm Optimization

- Task: Give AI a naive algorithm (e.g., $O(n^2)$ duplicate search).
- Instructions:
 - o Students write a brute force duplicate-finder.
 - o Ask AI to optimize it (e.g., by using sets/dictionaries with $O(n)$ time).
 - o Compare execution times with large input sizes.
- Expected Output:
 - o Two versions of the same algorithm (brute force + optimized).
 - o AI explanation of how complexity was improved.


```

▶ def find_duplicates_optimized(arr):
    """
    Finds duplicate elements in a list using a set for
    fast lookup.

    Args:
        arr: The list to search for duplicates.

    Returns:
        A list containing the duplicate elements found.
    """
    seen_elements = set()
    duplicates = []
    for element in arr:
        if element in seen_elements:
            duplicates.append(element)
        else:
            seen_elements.add(element)
    return duplicates

# Example usage:
my_list = [1, 2, 3, 4, 2, 5, 6, 3, 7, 8, 1]
duplicate_elements = find_duplicates_optimized(my_list)
print("Duplicate elements:", duplicate_elements)

```

```

⇒ Duplicate elements: [2, 3, 1]

```

```

import random

def generate_list_with_duplicates(size, num_duplicates):
    """
    Generates a list of a given size with a specified number of duplicates.

    Args:
        size: The total number of elements in the list.
        num_duplicates: The approximate number of duplicate values to add.

    Returns:
        A list of integers of the specified size containing duplicates.
    """
    if num_duplicates > size:
        raise ValueError("Number of duplicates cannot exceed the total size")

    # Create a list of unique elements
    unique_size = size - num_duplicates
    unique_elements = list(range(unique_size))

    # Generate duplicate values by randomly selecting from unique elements
    duplicate_values = [random.choice(unique_elements) for _ in range(num_duplicates)]

    # Combine unique elements and duplicate values
    combined_list = unique_elements + duplicate_values

    # Shuffle the combined list
    random.shuffle(combined_list)

    return combined_list

# Example usage:
test_list = generate_list_with_duplicates(20, 5)
print("Generated list with duplicates:", test_list)
print("Length of the list:", len(test_list))

```

Generated list with duplicates: [5, 9, 13, 2, 10, 12, 7, 2, 0, 3, 6, 1, 11, 8, 4, 14, 15, 16, 17, 18]
Length of the list: 20

```

import time

def find_duplicates_brute_force(arr):
    """
    Finds duplicate elements in a list using a brute-force approach.

    Args:
        arr: The list to search for duplicates.

    Returns:
        A list containing the duplicate elements found in the input list.
    """
    duplicates = []
    n = len(arr)
    for i in range(n):
        for j in range(i + 1, n):
            if arr[i] == arr[j] and arr[i] not in duplicates:
                duplicates.append(arr[i])
                break # Found one duplicate of arr[i], move to the next element
    return duplicates

list_sizes = [1000, 10000, 50000]
num_duplicates = 100

for size in list_sizes:
    print(f"\n--- Testing with list size: {size} ---")

    # Generate test data
    test_list = generate_list_with_duplicates(size, num_duplicates)

    # Measure Brute-Force method
    start_time = time.time()
    duplicates_brute_force = find_duplicates_brute_force(test_list)
    end_time = time.time()
    brute_force_time = end_time - start_time
    print(f"    Brute-Force Time: {brute_force_time:.6f} seconds")

    # Measure Optimized method
    start_time = time.time()
    duplicates_optimized = find_duplicates_optimized(test_list)
    end_time = time.time()
    optimized_time = end_time - start_time
    print(f"    Optimized Time: {optimized_time:.6f} seconds")

    # Optional: Verify that both methods found the same duplicates
    # print(f"    Brute-Force found {len(duplicates_brute_force)} duplicates.")
    # print(f"    Optimized found {len(duplicates_optimized)} duplicates.")
    # print(f"    Duplicates found match: {sorted(duplicates_brute_force) == sorted(duplicates_optimized)}")

```

```

--- Testing with list size: 1000 ---
    Brute-Force Time: 0.056865 seconds
    Optimized Time: 0.000161 seconds

```

```

--- Testing with list size: 10000 ---
    Brute-Force Time: 2.052830 seconds

```

Explain Improvement and Performance

The significant difference in execution times between the brute-force and optimized duplicate find methods is due to their underlying time complexities.

Brute-Force (`find_duplicates_brute_force`):

- **Approach:** This method uses nested loops. The outer loop iterates through each element, and the inner loop iterates through each subsequent element for a match. Additionally, it checks if the duplicate is already in the `seen_elements` set. This results in a quadratic time complexity.
- **Time Complexity:** $O(n^2)$ in the worst case. For each element in the list, the inner loop might iterate through all remaining elements. As the list size (`n`) increases, the number of operations grows quadratically. This results in a significant increase in time as the list size grows from 1,000 to 50,000.

Optimized (`find_duplicates_optimized`):

- **Approach:** This method uses a `set` to keep track of elements that have been seen so far. For each element in the list, it checks if the element is already in the `seen_elements` set. If it is, a duplicate is found. If not, the element is added to the set. This method uses hash tables, which allow for average-case $O(1)$ (constant time) lookups and insertions.
- **Time Complexity:** $O(n)$ on average. The algorithm iterates through the list once. For each element, checking if it is in the `seen_elements` set and adding it to the set takes, on average, constant time. Therefore, the total time complexity is linear with respect to the number of elements in the list.

Performance Differences:

The performance results clearly demonstrate the impact of the time complexities:

- For smaller list sizes (e.g., 1,000), the difference is noticeable but not drastic.
- As the list size increases (e.g., 10,000 and 50,000), the optimized method becomes orders of magnitude faster than the brute-force method. The brute-force method takes seconds for a list of 50,000 elements, while the optimized method takes milliseconds.

This stark difference highlights the importance of choosing the right data structure and algorithm for a given problem. By using a set, the algorithm transforms the problem from a quadratic time complexity to a linear time complexity, resulting in significant performance improvements for large datasets.